SECT-4451 — Lecture 2 Practical: Data Exploration & Foundational Math

Task: Instead of creating your own data, use the provided dataset. This data represents student performance, but it contains a "dirty" record and significant variance in scale.

Part A — Inspect and Clean

1. Identify the outlier in study_hours (the student with 50 hours). Remove or cap this value as you see fit. Justify your choice.
2. What is the shape of the feature matrix? Which columns are numerical? Which column is your target variable?

Part B — Summarize

1. Calculate the mean, median, and standard deviation for all columns.
2. Write 3 observations in plain English. Example: "The standard deviation of attendance is low, suggesting that most students maintain a consistent attendance rate."

Part C — Visualize

1. Create a histogram for final_score.
2. Create a scatter plot of study_hours vs final_score. Describe the trend.

Part D — Correlation and Causality

1. Calculate the Pearson correlation coefficient between study_hours and final_score.
2. Critical Thinking: If a student studies 10 hours, does the correlation guarantee they will get a specific score? What is a potential "lurking" or "confounding" variable that could explain the link between study time and scores?

Part E — Vectors and Dot Product

1. Select one student row (excluding the target) as a vector \mathbf{x}.
2. Define a weight vector f{w} = [0.5, 0.2, 0.4] and a bias b = 5.
3. Calculate {y} = f{w} dot f{x} + b using NumPy.
4. Conceptually: What does f{w} represent in the context of predicting a score? Why do we need b?

Part F — Scaling

1. Standardize study_hours and previous_mark (use (x -mean) / standard degviation).
2. Why would a distance-based algorithm (like K-Nearest Neighbors) fail if we didn't scale these two features? (Hint: compare the original range of study_hours vs previous_mark).

Code for dataset:

```
import pandas as pd
import numpy as np


np.random.seed(42)


n = 100
study_hours = np.random.normal(5, 2, n)
attendance = np.random.normal(85, 10, n)
previous_mark = np.random.normal(70, 15, n)

final_score = (0.5 * study_hours) + (0.3 * attendance) + (0.2 * previous_mark) +
np.random.normal(0, 5, n)

df = pd.DataFrame({
    'study_hours': study_hours,
    'attendance': attendance,
    'previous_mark': previous_mark,
    'final_score': final_score
})


df.loc[0:5, 'study_hours'] = np.nan  # Missing values
df.loc[99, 'study_hours'] = 50       # Outlier
df.loc[98, 'attendance'] = -10       # Impossible value
```

```
print(df)
# df.to_csv('student_data.csv', index=False)
```